

Lekce 8: Práce s asercemi a validacemi

1. Osnova obsahu

A. Úvod do asercí

- **Definice:**
 - Aserce jsou příkazy ve vašem testovacím kódu, které ověřují, zda je splněna určitá podmínka. Jsou zásadní pro ověření, že stav aplikace odpovídá očekávaným výsledkům.
- **Proč jsou aserce důležité:**
 - **Detekce chyb:** Pomáhají rychle rozpoznat, kdy se aplikace nechová podle očekávání.
 - **Spolehlivost testů:** Zajišťují, že změny v kódu nenaruší očekávanou funkcionalitu.
 - **Dokumentace:** Slouží jako živá dokumentace toho, co by aplikace měla dělat.

B. Aserce v Cypressu a integrace s Chai

- **Vestavěné aserce v Cypressu:**
 - Cypress používá pod kapotou Chai pro aserce.
- **Styly asercí v Chai:**
 - **Should:** Řetěžitelné aserce pomocí `should()` .
 - **Expect:** Funkcionální styl asercí pomocí `expect()` .
 - **Assert:** Klasický styl asercí pomocí `assert` .
- **Příklady:**
 - `cy.get(selector).should('be.visible')`
 - `expect(value).to.equal(expectedValue)`

Definice:

Chai je populární knihovna pro aserce v JavaScriptu, která nabízí různé styly psaní testů. Umožňuje vývojářům psát čitelné testy s jasnými a popisnými asercemi.

Jak používat Chai:

- **Integrace s Cypress:**

Cypress má Chai již zabudované, takže jeho aserce lze používat přímo v testech bez další konfigurace.
- **Hlavní styly asercí v Chai:**
 - **Should styl:**

Používá řetěžitelný jazyk pro sestavování asercí čtivých jako přirozený jazyk.

```
cy.get('[data-testid="login-button"]').should('be.visible');
```

- **Expect styl:**

Ověřuje očekávání pomocí volání funkcí.

```
cy.get('[data-testid="login-button"]').then(($btn) => {  
  expect($btn).to.be.visible;  
});
```

- **Assert styl:**

Používá klasické asertovací funkce.

```
cy.get('[data-testid="login-button"]').then(($btn) => {  
  assert.isTrue($btn.is(':visible'), 'Login button is visible');  
});
```

Nejčastěji používané aserce v Cypressu

1. `should('exist')`

Ověřuje, že prvek existuje v DOMu.

```
cy.get('[data-testid="login-button"]').should('exist');
```

Použijte, když: Chcete ověřit, že prvek byl vykreslen.

2. `should('be.visible')`

Ověřuje, že je prvek **viditelný** uživateli.

```
cy.get('[data-testid="modal"]').should('be.visible');
```

Použijte, když: Chcete mít jistotu, že uživatel prvek vidí/může s ním interagovat.

3. `should('not.exist')` / `should('not.be.visible')`

Opak předchozího – vhodné pro testování **odstranění nebo skrytí** prvků.

```
cy.get('[data-testid="loading-spinner"]').should('not.exist');
```

4. `should('have.text', 'nějaký text')`

Ověřuje **přesný** textový obsah v prvku.

```
cy.get('[data-testid="welcome-message"]').should('have.text', 'Welcome back!');
```

Alternativa:

Použijte `contains()` pro částečnou shodu, nebo `.should('include.text', 'Welcome')`.

5. `should('include.text', 'část textu')`

Ověřuje, že prvek obsahuje podřetězec (není vyžadována přesná shoda).

```
cy.get('.alert').should('include.text', 'Error');
```

6. `should('have.value', 'text')`

Ověřuje hodnotu vstupních polí.

```
cy.get('[data-testid="email-input"]').should('have.value', 'user@example.com');
```

Použijte, když: Chcete potvrdit, že pole bylo vyplněno správně.

7. `should('be.checked')` / `should('not.be.checked')`

Používá se pro checkboxy nebo rádio tlačítka.

```
cy.get('#terms-checkbox').should('be.checked');
```

8. `should('be.disabled')` / `should('be.enabled')`

Zajišťuje, že formulářové prvky jsou (ne)aktivní podle očekávání.

```
cy.get('[data-testid="submit-btn"]').should('be.disabled');
```

9. `should('have.class', 'class-name')`

Ověřuje, že má prvek určitou třídu.

```
cy.get('button').should('have.class', 'active');
```

10. `should('have.attr', 'attribute', 'value')`

Ověřuje, že prvek má specifický atribut s danou hodnotou.

```
cy.get('a').should('have.attr', 'href', '/dashboard');
```

Také užitečné pro kontrolu přítomnosti atributů `data-*` nebo `target`, `src`, `disabled` atd.

11. `should('have.length', číslo)`

Ověření počtu prvků vrácených pomocí `cy.get()`.

```
cy.get('.list-item').should('have.length', 5);
```

Použijte, když: Chcete zkontrolovat, zda seznam obsahuje očekávaný počet prvků.

12. `should('match', /regex/)`

Ověřuje, že řetězec odpovídá regulárnímu výrazu.

```
cy.get('[data-testid="email"]').invoke('text').should('match', /^[^\s@]+@[^\s@]+\.[^\s@]
```

Používejte `expect()` uvnitř bloku `.then()` při práci s logikou nebo vlastními daty:

```
cy.get('#price').then(($el) => {  
  const price = parseFloat($el.text().replace('€', ''));  
  expect(price).to.be.lessThan(100);  
});
```

- **Should styl:**

- **Výhody:**

- Velmi čitelný a výstižný (např. `should('be.visible')`).
 - Podporuje řetězení více asercí přirozeným jazykem.
 - Automaticky opakuje aserce při použití s příkazy Cypressu.

- **Nevýhody:**
 - Požaduje rozšíření prototypů objektů, což nemusí být v každém prostředí vhodné.
- **Expect styl:**
 - **Výhody:**
 - Jasný funkcionální styl, který je vývojářům povědomý.
 - Není třeba rozšiřovat prototypy.
 - **Nevýhody:**
 - Při použití s příkazy Cypressu se aserce automaticky neopakuje, protože je obvykle volán uvnitř `.then()` bloku.
- **Assert styl:**
 - **Výhody:**
 - Tradičnější a přímočařejší, podobné asercím v jiných jazycích.
 - **Nevýhody:**
 - Méně čitelné a vyžaduje více šablonového kódu.

C. Běžné asertovací metody v Cypressu

- **Viditelnost a existence:**
 - `should('be.visible')` , `should('exist')` , `should('not.exist')`
- **Ověření obsahu:**
 - `should('have.text', 'očekávaný text')`
 - `should('contain', 'část textu')`
- **Kontrola atributů:**
 - `should('have.attr', 'názevAtributu', 'hodnota')`
- **Kontroly CSS a stylů:**
 - `should('have.css', 'vlastnost', 'hodnota')`
- **Stavové kontroly:**
 - `should('be.disabled')` , `should('be.enabled')` , `should('be.checked')`

D. Pokročilé aserce

- **Vícenásobné aserce:**
 - Řetězte více příkazů `should()` pro komplexní validaci.
 - Příklad: Ověření, že prvek je viditelný a obsahuje konkrétní text.
- **Podmíněné aserce:**
 - Použijte `.then()` , chcete-li provádět aserce na základě dynamického obsahu nebo podmínek.
- **Vlastní aserce:**
 - Definujte vlastní asertovací logiku v případě, že vestavěné nestačí.

Vícenásobné aserce a řetězení asercí

Řetězení asercí pomocí `should()` :

- **Příklad:**

```
cy.get('[data-testid="username-input"]')
  .should('be.visible')
  .and('have.attr', 'placeholder', 'Enter your username')
  .and(($input) => {
    // Vlastní kontrola, že vstup je ve výchozím stavu prázdný
    expect($input.val()).to.equal('');
  });
```

- **Vysvětlení:**

Tento řetězec:

- Ověří, že prvek je viditelný.
- Zkontroluje, že atribut placeholder má očekávanou hodnotu.
- Proveďte vlastní asertci, že pole je prázdné.

Vícenásobné aserce v jednom příkazu:

- Používání více volání `should()` nebo řetězení pomocí `and()` umožňuje ověřit více podmínek na jednom prvku bez opakování dotazování na DOM, což je efektivní a zlepšuje čitelnost.

Proč používat `should()` namísto `expect()` v Cypressu

- **Automatický retry mechanismus:**

- `should()` je integrován s Cypress opakováním. Pokud aserce selže kvůli dočasnému stavu (například prvek se teprve načítá), Cypress se automaticky pokusí aserci zopakovat až do splnění nebo uplynutí timeoutu.
- `expect()` se používá uvnitř `.then()` bloků, a proto se provede pouze jednou. Když aktuálně prvek není ve správném stavu, test okamžitě selže.

- **Řetězení:**

- `should()` umožňuje řetězit více asercí nad stejným objektem, tím se snižuje počet dotazů na DOM a testy jsou stručnější.

- **Čitelnost:**

- Plynulý, přirozený styl `should()` zvyšuje čitelnost a rychlost pochopení asercí.

- **Konzistence s příkazy Cypressu:**

- Použitím `should()` přímo na Cypress příkazech se aserce hladce integrují s Cypress command queue a jsou automaticky opakovány podle potřeby.

Ukázka porovnání:

Použití `should()` (doporučeno):

```
cy.get('[data-testid="submit-button"]')
  .should('be.visible')
  .and('not.be.disabled');
```

Použití `expect()` (méně vhodné v kontextu Cypressu):

```
cy.get('[data-testid="submit-button"]').then(($btn) => {
  expect($btn).to.be.visible;
  expect($btn).to.not.be.disabled;
});
```

- Ve druhém příkladu, pokud tlačítko není ihned viditelné, test okamžitě selže, bez opakování.

E. Osvědčené postupy pro aserce a validace

- **Jasně a popisné aserce:**

- Pište aserce, které přesně vysvětlují, co ověřují.
- V případě potřeby používejte vlastní chybové zprávy nebo logování.
- Pište aserce, které jasně uvádějí, co očekáváte.

Příklad:

```
cy.get('[data-testid="error-message"]')
  .should('be.visible')
  .and('contain', 'Invalid credentials');
```

- Vyhněte se nejasným nebo příliš složitým asercím, které kombinují více ověření dohromady bez jasné separace.

- **Granulární testování:**

- Ověřujte jednu podmínku na aserci, když to lze. To usnadňuje nalezení chyb.
- Preferujte ověřování jedné podmínky na aserci, abyste oddělili případné chyby.
Pokud však podmínky souvisejí (například viditelnost a obsah), použijte řetězení s `should()`.

- **Vyhýbejte se překrývajícím se asercím:**

- Dbejte na to, aby aserce byly logicky oddělené, což usnadňuje odhalení konkrétní chyby.
- Neověřujte stejnou podmínku vícekrát v jednom kroku testu; každá aserce by měla mít unikátní účel.

- **Využívejte opakovací mechanismus Cypressu:**

- Cypress automaticky opakuje aserce, dokud neprojdou nebo neuplyne timeout, proto pište aserce odolné vůči časování.
- **Správné řetězení:**
 - Řetězte aserce pro snížení opakování kódu a zlepšení čitelnosti.

2. Ukázky kódu

A. Základní aserce pomocí `should`

```
// Ověřte, že tlačítko pro přihlášení je viditelné a aktivní
cy.get('[data-testid="login-button"]')
  .should('be.visible')
  .and('not.be.disabled');
```

B. Ověření textového obsahu

```
// Ověřte, že zpráva o úspěchu obsahuje očekávaný text
cy.get('[data-testid="success-message"]')
  .should('be.visible')
  .and('contain', 'Login successful');
```

C. Kontrola atributů prvku

```
// Ověřte, že obrázek má správný src atribut
cy.get('[data-testid="hero-image"]')
  .should('have.attr', 'src', 'images/hero.jpg');
```

D. Použití `expect` pro aserce

```
// Použití expect pro aserce na proměnných
cy.get('[data-testid="user-count"]').then(($count) => {
  const countText = $count.text();
  expect(parseInt(countText)).to.be.greaterThan(0);
});
```


E. Řetězení více asercí

```
// Ověření více podmínek na formulářovém vstupu pomocí řetězení
cy.get('[data-testid="username-input"]')
  .should('be.visible')
  .and('have.attr', 'placeholder', 'Enter your username')
  .and(($input) => {
    // Vlastní aserce: pole není prázdné po vyplnění
    expect($input.val()).to.not.be.empty;
  });
```

F. Podmíněné aserce s .then()

```
// Použití .then() k podmíněnému ověření na základě dynamického obsahu
cy.get('[data-testid="error-message"]').then(($el) => {
  if ($el.is(':visible')) {
    expect($el).to.contain('Invalid credentials');
  } else {
    cy.log('Chybová zpráva není vidět, test možná prošel.');
  }
});
```

3. Potenciální studentské dotazy

1. Jaký je rozdíl mezi `should` a `expect` v Cypressu?

- **Odpověď:**

`should()` je řetěžitelný příkaz, který se automaticky opakuje, dokud aserce neprojde, zatímco `expect()` se používá pro jednorázové aserce uvnitř bloku `.then()`.

2. Jak Cypress pracuje s asercemi, když načítání prvků trvá déle?

- **Odpověď:**

Cypress automaticky opakuje aserce, dokud aserce neprojde nebo nevyprší defaultní timeout, což umožňuje hladkou práci s asynchronními prvky.

3. Mohu psát vlastní aserce v Cypressu?

- **Odpověď:**

Ano, můžete psát vlastní aserce uvnitř bloků `.then()` pro složitější validační logiku.

4. Proč se vyhýbat překrývajícím se asercím?

- **Odpověď:**

Překrývající se aserce ztěžují identifikaci, která podmínka selhala. Oddělené aserce usnadňují ladění a údržbu testů.

5. Co se stane, když aserce selže?

- **Odpověď:**

Cypress přestane test vykonávat a vypíše chybu včetně detailů o selhání, včetně screenshotů a logů pro ladění.

4. Další doporučení

- **Interaktivní ladění:**

- Doporučte studentům použití interaktivního Test Runneru Cypress pro sledování, jak se aserce opakují.

- **Skupinová cvičení:**

- Pracujte ve dvojicích a pište testovací případy s více asercemi a diskutujte logiku každé z nich.

- **Studium dokumentace:**

- Nechte studenty prozkoumat [Cypress Assertions Documentation](#) pro další studium.